

Google Ads API — Business Model & Use Case

PhoneSpot | Website: <https://www.phonespot.co.kr> | Manager (MCC): 396-470-5146 | v2 (resubmission)

1. Company & Business Model

PhoneSpot (■■■■, <https://www.phonespot.co.kr>) is a mobile-phone retail business in South Korea, operating physical stores that sell smartphones, mobile carrier plans, and accessories. Revenue comes from device sales and new carrier-plan activations. Customers are acquired primarily online: the company runs paid advertising on Google Ads, Meta (Facebook/Instagram), Naver, and Daangn, driving prospective customers to inquire via KakaoTalk chat and phone calls, who then complete a purchase/activation at a store.

Note on website: www.phonespot.co.kr is a live, client-rendered (JavaScript) site whose homepage title is “■■■■ - ■■■■ ■■■■■■” (“PhoneSpot — find your neighborhood phone deal”). If the review crawler renders an empty shell, the business description in this document represents the site's content and operations.

2. Why We Need the Google Ads API (Use Case)

We measure advertising efficiency by combining ad spend with downstream conversions (KakaoTalk inquiries, calls, store activations). Today, Google Ads metrics are copied manually into our internal Google Sheets dashboard, which is slow and error-prone. We want to automatically pull our own Google Ads reporting metrics once per day so the dashboard always shows up-to-date cost-per-inquiry (CPL) per campaign and ad group, alongside Meta/Naver/Daangn and GA4 conversion data.

The tool is **read-only reporting** for accounts we own. It does **not** create, edit, pause, or manage campaigns, budgets, or creatives, and is used only by our internal staff.

3. System Architecture

- 1 A Google Apps Script (bound to our internal Google Sheets) runs on a daily time-based trigger.
- 2 It exchanges a stored OAuth2 refresh token for a short-lived access token.
- 3 It issues one GAQL query via `GoogleAdsService.SearchStream` for our own ad account.
- 4 Returned daily ad-group rows (date, campaign, ad group, impressions, clicks, cost) are written into the sheet.
- 5 The sheet combines them with GA4 conversion data to compute CPL.

4. API Usage (read-only)

Item	Detail
Service / Method	<code>GoogleAdsService.SearchStream</code> ; <code>CustomerService.listAccessibleCustomers</code> (setup only)
Resources	<code>ad_group</code> , <code>campaign</code> , <code>customer</code>
Metrics	<code>metrics.impressions</code> , <code>metrics.clicks</code> , <code>metrics.cost_micros</code>
Segments	<code>segments.date</code>
Mutations	None (no create/update/remove)

Call volume	~1 reporting request/day (far below Basic limit)
-------------	--

5. Mock-up of the Internal Tool (Dashboard)

The API data lands in our internal “■■■■■■■” (Unified Dashboard) sheet. Representative layout the API populates (Google row auto-filled by this tool):

Date	Channel	Campaign / Ad group	Impr.	Clicks	Cost (KRW)	Inquiries	CPL
2026-06-21	Google	Search / brand	12,340	210	85,000	4	21,250
2026-06-21	Meta	Reels / promo	30,120	540	120,000	7	17,142
2026-06-21	Naver	PowerLink / phone	8,900	160	60,000	3	20,000

(Numbers above are illustrative. The Google row's Impr./Clicks/Cost are the fields fetched via the API; Inquiries/CPL are computed from GA4 + internal records.)

6. Access, Security & Compliance

OAuth2 credentials (developer token, client ID/secret, refresh token) are stored server-side in Apps Script Script Properties, never exposed to users. Access is limited to our own MCC (396-470-5146) and the ad accounts it manages. Internal employees only; no external or public users; no conversion upload or remarketing. The tool is used solely for read-only performance reporting of accounts we own, consistent with the Google Ads API Terms.